Практична робота № 4.

Тема: Алгоритми сортування та фільтрації багатовимірних масивів

Мета: Ознайомитися з особливостями збереження даних у багатовимірних індексованих масивах

ТЕОРЕТИЧНІ ВІДОМОСТІ

Доступ до даних

У багатовимірному індексованому масиві для отримання значення використовується ланцюжок квадратних дужок:

`$array[рядок][стовпець].`

Наприклад, `$matrix[0][2]` звернеться до першого вкладеного масиву та його третього елемента.

Сортування.

Сортувати можна як прості, так і асоціативні масиви. Для сортування масивів в PHP існують певні функції:

• sort() - сортує масив в алфавітному порядку, якщо хоч би один з його елементів є рядком, і в числовому порядку, якщо усі його елементи - числа.

• rsort() - працює як sort( ), але в зворотному порядку.

• asort() - сортує асоціативний масив; працює як sort( ), але зберігає імена елементів.

• arsort()- працює як asort( ), але в зворотному порядку.

• ksort() - сортує асоціативний масив по іменах елементів.

• krsort()- працює як ksort( ), але в зворотному порядку.

Ключові функції для обробки

- `array_column(array, index)` – Надзвичайно корисна функція, що дозволяє "витягнути" цілий стовпець із двовимірного масиву за його порядковим номером.
- `array_map(callback, ...arrays)` – Дозволяє застосувати функцію до кожного елемента масиву або об’єднати кілька масивів (наприклад, для віднімання одного стовпця від іншого).
- `array_sum()` –Швидко обчислює суму всіх числових значень у масиві.
- `usort(array, callback)` – Сортує масив, де логіку порівняння (наприклад, за сумою елементів рядка) ви пишете самі.

Приклад практичного завдання.

Умова: Дано індексований масив, що представляє таблицю результатів 4-х забігів для 3-х атлетів. Кожен рядок — це результати одного атлета (час у секундах за кожен забіг).

```
$results = [
```

```
     [12.5, 11.8, 12.2, 11.5], // Атлет 0
     [13.1, 12.9, 12.8, 13.0], // Атлет 1
     [11.9, 12.1, 11.7, 12.3], // Атлет 2
 ];
```

Завдання.

Агрегація: Для кожного атлета обчислити його середній час за всі забіги. Результати вивести у вигляді нового одновимірного масиву.

Пошук: Знайти "найкращий забіг" (мінімальне число) у всій таблиці та вивести його значення та координати (номер атлета та номер забігу).

Трансформація: Створити "вектор прогресу", який є різницею між результатами останнього (індекс 3) та першого (індекс 0) забігів для кожного атлета.

Сортування: Відсортувати атлетів у масиві $results за їхнім найкращим часом (хто пробіг хоча б один раз найшвидше — той перший).

Шаблон виконання

```
<?php

/**
 * Вхідні дані: Результати 4-х забігів для 3-х атлетів (час у
секундах).
 * Структура: [Атлет][Забіг]
 */
$results = [
    [12.5, 11.8, 12.2, 11.5], // Атлет №0
    [13.1, 12.9, 12.8, 13.0], // Атлет №1
    [11.9, 12.1, 11.7, 12.3], // Атлет №2
];

echo "<h3>Вихідна таблиця результатів:</h3>";
echo "<pre>" . print_r($results, true) . "</pre>";

// --- ЗАВДАННЯ 1: СЕРЕДНІЙ ЧАС ---
// Створіть масив $averages, де кожен елемент - це середнє
значення рядка.
// Використовуйте array_map та array_sum.

$averages = array_map(fn($athlete) => array_sum($athlete) /
count($athlete), $results);

echo "<h4>Середні результати атлетів:</h4>";
print_r($averages);


// --- ЗАВДАННЯ 2: ПОШУК РЕКОРДУ ---
// Знайдіть найшвидший забіг (мінімальне число) у всій матриці.
// Виведіть значення та його "координати" [атлет][забіг].

$minVal = $results[0][0];
```

```
  $coord = [0, 0];

  foreach ($results as $i => $row) {
      foreach ($row as $j => $val) {
          if ($val < $minVal) {
              $minVal = $val;
              $coord = [$i, $j];
          }
      }
  }

  echo "<h4>Найкращий результат: $minVal (Атлет №{$coord[0]},
Забіг №{$coord[1]})</h4>";


  // --- ЗАВДАННЯ 3: ВЕКТОР ПРОГРЕСУ ---
  // Створіть масив, що показує різницю між останнім (індекс 3) та
першим (індекс 0) забігом.
  // Використовуйте array_column.

  $firstHeat = array_column($results, 0);
  $lastHeat  = array_column($results, 3);
  $progress  = array_map(fn($start, $end) => round($end - $start,
2), $firstHeat, $lastHeat);

  echo "<h4>Прогрес (останній забіг мінус перший):</h4>";
  print_r($progress);


  // --- ЗАВДАННЯ 4: СОРТУВАННЯ ---
  // Відсортуйте масив $results так, щоб атлет з найкращим
ОДИНИЧНИМ результатом був першим.
  // Використовуйте usort та min().

  usort($results, fn($a, $b) => min($a) <=> min($b));

  echo "<h4>Таблиця після сортування за найкращим особистим
результатом:</h4>";
  echo "<pre>" . print_r($results, true) . "</pre>";
```

Методичні вказівки до виконання

Функція `array_column`. Дозволяє працювати з "вертикальними" даними в матриці. Пам'ятайте, що в PHP масиви за замовчуванням "горизонтальні" (рядкові).

Оператор <=> (`Spaceship`). Використовується в `usort`. Якщо $a < $b, повертає -1, якщо однакові – 0, якщо $a > $b – 1.

Стрілкові функції (`fn() => ...`). Використовуйте їх для коротких операцій. Вони автоматично мають доступ до змінних із зовнішньої області видимості.

ЗАДАЧІ ДЛЯ САМОСТІЙНОГО ВИКОНАННЯ

Блок 1: Одновимірні масиви (Рівень: Базовий*)*

Задання 1. Стипендіальний фонд.

Дано масив балів 16 студентів. Використовуючи array_filter, видалити всі бали нижче 60. Для решти обчислити середній бал за допомогою array_sum та count.

Завдання 2. Футбольна таблиця.

Маємо масив результатів (0, 1, 3).

- Порахувати кількість перемог, використовуючи array_count_values.
- Знайти індекси (номери) команд з максимальним результатом, використовуючи array_keys(array, max(array)).

Блок 2: Двовимірні масиви (Рівень: Алгоритмічний)

Завдання 3. Операції на матрицею.

Задано матрицю цілих чисел `A[n][n]`.

- *Аналіз стовпців:* За допомогою `array_column` виділити 2-й та 3-й стовпці. Створити новий масив (вектор), який є їх різницею (використати `array_map`).
- *Пошук екстремумів:* Знайти координати $[i][j]$ мінімального та максимального елементів матриці. Поміняти їх місцями.

Підказка: Використовуйте `foreach` для ітерації та збереження координат.

- **Глобальна заміна**: Замінити всі елементи, що дорівнюють числу X, на число 99. Реалізувати це через array_walk_recursive та анонімну функцію (Closure).

ДОДАТКОВІ ЗАДАЧІ

1. Найти кількість різних елементів масиву.
2. В масиві є два однакових елементи. Знайти їх та вивести на екран.
3. Дано масив розміру $N \geq 2$ і число k ($0 < k < N$). Здійснити циклічний зсув елементів масиву вправо на k позицій.
4. Дано масив натуральних чисел розміру N. З'ясувати чи є він перестановкою чисел від 1 до N. Вивести YES, якщо масив є деякою перестановкою чисел від 1 до N, в іншому випадку вивести NO.